

Practical-Cryptography: exercises

Sofia Celi

September 2026

1 Lecture 1 & 2

How to clone:

```
GIT_SSH_COMMAND="ssh -i /path/to/repo_deploy_key -o IdentitiesOnly=yes"
git clone git@github.com:OWNER/REPO.git''
```

or set up an entry in `/.ssh/config` pointing at that key.

Goals

In the lectures we saw how single-server PIR with preprocessing works on paper. In this tutorial you will run a real implementation of *FrodoPIR*, measure it, push it until it breaks, and compare what you observe with what the theory predicts. By the end you should be able to answer, with your own numbers: *what does a private query cost, what does that cost depend on, and when is it worth paying?*

Throughout, we use the notation from the lectures: the database has m records, each record is split into ω chunks of $\log \rho$ bits, n is the LWE dimension, and q is the ciphertext modulus. The database is the matrix $D \in \mathbb{Z}_\rho^{m \times \omega}$, the public matrix is $A \in \mathbb{Z}_q^{n \times m}$, and the hint is $M = A \cdot D \in \mathbb{Z}_q^{n \times \omega}$.

Part 0: Setup

1. Install Rust (<https://rustup.rs>) and check that `cargo --version` works.
2. Clone the course repository and enter the tutorial:

```
git clone https://github.com/claucece/Practical-Cryptography
cd Practical-Cryptography/tutorials/frodo-pir-fast
```

3. Build it

```
make build
```

Part 1: run it, read it, measure it

Exercise 1: run the tests.

`make test`

The main test runs a full client-server interaction several times and checks that the client recovers the right record.

- (a) Which test does this? What database size and record size does it use?
- (b) Run the tests several times. Do they always pass? If one fails occasionally, keep a note of it: you will explain why in Exercise 6.

Exercise 2: map the code to the protocol. Find the code for each of the following steps. For each one, write down the file, the function, and whether it runs on the client or the server, and whether it is *offline* (once) or *online* (per query).

- (a) Encoding the database as the matrix D (how is a record split into ω chunks?).
- (b) Deriving A from a short seed. Why is a seed sent instead of A itself?
- (c) Computing the hint $M = A \cdot D$.
- (d) Building the query $b = s^T A + e^T + f_i$, where f_i is q/ρ in position i and 0 elsewhere.
- (e) The server response $c = b \cdot D$.
- (f) Decoding: $c - s^T M$, then rounding to the nearest multiple of q/ρ .

Which part of (d) does not depend on the index i ? What does the implementation do with that observation?

Exercise 3: first benchmarks.

`make bench`

The parameters are set at the top of the `Makefile` and are all exponents or plain integers:

Variable	Default	Meaning
MATRIX_HEIGHT_EXP	16	$m = 2^{16}$ records
ELEMENT_SIZE_EXP	13	records of 2^{13} bits = 1 KB
PLAINTEXT_SIZE_EXP	10	$\rho = 2^{10}$, i.e. 10 bits per matrix entry
LWE_DIMENSION	1774	n

The benchmarks use Criterion; a browsable report is written to `target/criterion/report/index.html`. Fill in the following table for the default parameters.

Quantity	Who / when	Your measurement
Database preprocessing (compute M)	server, offline	
Hint size	download, offline	
Query preprocessing ($s^T A + e^T$)	client, offline	
Query generation	client, online	
Query size	upload, online	
Server response time	server, online	
Response size	download, online	
Decoding	client, online	

Note your machine (CPU, RAM) next to the table: you will compare with your neighbours if you want!

Exercise 4: predict before you measure. Before moving to Part 2, use the formulas from the lecture to *predict* the following for $m = 2^{16}$ and $m = 2^{20}$ records of 1 KB each, with $q = 2^{32}$:

- (a) the query size ($m \log q$ bits),
- (b) the response size ($\omega \log q$ bits),
- (c) the hint size ($n \omega \log q$ bits),
- (d) the number of multiplications the server performs per query ($m \cdot \omega$).

Keep your predictions; you will check them next.

Part 2 (Lecture 2): scale it, break it, extend it

Exercise 5: scaling experiments. You can override any parameter on the command line, for example

```
make bench MATRIX_HEIGHT_EXP=18
make bench ELEMENT_SIZE_EXP=15
```

- (a) Fix the record size at 1 KB and vary `MATRIX_HEIGHT_EXP` in $\{10, 12, 14, 16, 18\}$.
- (b) Fix $m = 2^{16}$ and vary `ELEMENT_SIZE_EXP` in $\{11, 13, 15, 17\}$, i.e. records of 256 B, 1 KB, 4 KB and 16 KB.
- (c) If your machine has enough memory, run `make bench-all`, which covers $m = 2^{16}$ to 2^{20} and writes `benchmarks-16.txt` to `benchmarks-20.txt`. Before you do, estimate the memory the server needs for $m = 2^{20}$ records of 1 KB if every matrix entry is stored in 32 bits. Is that more or less than the raw database, and why?

Plot every row of the table from Exercise 3 against m and against the record size (log-log axes). Then answer:

- (i) Which quantities grow with m ? Which do not depend on m at all?
- (ii) Which quantities grow with the record size?
- (iii) Do the slopes match your predictions from Exercise 4? Where they do not, why (caches, memory bandwidth, parallelism, constant overheads)?
- (iv) The server touches every entry of D for every query. Compute your server's throughput in GB of database per second. How close is it to your machine's memory bandwidth?

Exercise 6: correctness and noise. Decoding is correct as long as the noise satisfies

$$|e^T \cdot D_{*,k}| < \frac{q}{2\rho} \quad \text{for every column } k.$$

- (a) Find where ρ (the number of plaintext bits per matrix entry) is set. Increase it step by step for a fixed m . At which value does decoding start to fail? How often?
- (b) Repeat for a larger m . Does the failure point move? In which direction, and why?
- (c) Explain the occasional test failure from Exercise 1(b), if you saw one.
- (d) What do you gain by increasing ρ when it *does* work? (Look at ω , the response size and the hint size.)

Exercise 7: misuse. Reusing query randomness. The library refuses to reuse the same query parameters twice. Find the test that checks this.

- (a) Suppose a client sends two queries $b_1 = s^T A + e^T + f_i$ and $b_2 = s^T A + e^T + f_j$ with the same s and e . What does the server learn from $b_1 - b_2$?
- (b) What if only s is reused and e is fresh? (Hint: compare the size of e with q/ρ .)
- (c) Optional: disable the check, implement the attack on the server side, and recover both indices.

Exercise 8: what does the client learn? PIR protects the client's query. It does not protect the database.

- (a) Modify the client so that the query contains q/ρ in *two* positions i and j . What does the client decode? Can the server tell that the query was malformed?
- (b) The hint $M = A \cdot D$ is public. Does it reveal anything about records the client never queries? Could a client recover D from M when $n \geq m$? What about $n < m$?
- (c) Name one application where this leakage is harmless and one where it is not. What primitive would you need in the second case?

Exercise 9: is it worth it? The trivial PIR is to download the whole database.

- (a) For $m = 2^{20}$ records of 1 KB, compute the total communication of FrodoPIR for Q queries (hint plus Q queries and responses) and compare it with downloading the database once. For which Q does FrodoPIR use less bandwidth?
- (b) Bandwidth is not the only cost. List three reasons why a deployment might still prefer PIR beyond that point (think of client storage, database updates, and mobile clients).
- (c) Using your server timing and the price of a cloud instance of your choice, estimate the cost in dollars of answering one million queries.

Exercise 10: extensions (pick one).

- (a) **The square trick.** FrodoPIR's upload is linear in m . Reshape the database as a $\sqrt{N} \times \sqrt{N}$ matrix as in SimplePIR: the query selects a column, the client keeps one row of the answer. Implement it (or work it out on paper) and compare upload, download and hint size with the original layout.
- (b) **Keyword queries.** A query with three non-zero positions returns the sum of three rows (Exercise 8a). Explain how this, together with a Binary Fuse filter that satisfies $F[h_1(k)] + F[h_2(k)] + F[h_3(k)] = \text{value}(k) \bmod \rho$, gives keyword PIR with no change to the server (this is ChalametPIR). What is the overhead in the number of rows? Can you extend it to use <https://github.com/claucece/chalamet?>
- (c) **Database updates.** One record of D changes. Which part of M changes? Give the cost of updating M on the server and the amount of data a client must download to refresh its hint, and compare with recomputing everything.
- (d) **Make it faster.** Profile the server response. Try one optimisation (multi-threading, a different memory layout for D , smaller integer types for the database entries) and report the speed-up.